Amazon Route 53 — Complete Notes

Scalable DNS & domain registration · Connect names to resources

A reader-friendly, all-in-one reference for Amazon Route 53.

1. 🌐 What is Route 53

Amazon Route 53 is a **highly available, scalable DNS web service**. It does three core jobs: **register domain names**, **route internet traffic** to your resources by translating names (like `example.com`) into IP addresses, and **check the health** of your resources so traffic only goes to healthy endpoints.

> 💡 **Mental model:** Route 53 is the **phone book of the internet for your app** — it answers "what address does this name point to?" and can answer **differently** based on geography, latency, weights, or health. The name "53" comes from **DNS port 53**.

Key facts: global service (not region-scoped), backed by a **100% availability SLA**, and deeply integrated with AWS (ALB, CloudFront, S3, etc.).

2. 🧩 Core Concepts

Concept	Description
Domain	A human-readable name you register (`example.com`).
Hosted Zone	A container for all DNS records of one domain.
Record (Record Set)	A single DNS entry (name → type → value), e.g. `www → A → 1.2.3.4` .
Name Server (NS)	The authoritative servers that answer for your zone.
TTL	Time-to-live — how long resolvers cache a record (seconds).
Routing Policy	The rule deciding *which* answer to return.
Health Check	A probe that tests whether an endpoint is healthy.
Resolver	The component that answers DNS queries (incl. hybrid on-prem).
Alias Record	A Route 53-specific record pointing to an AWS resource.

3. 🔎 How DNS Resolution Works

When a user visits `www.example.com`, this chain resolves the name to an IP:

```
Browser
   │  "www.example.com?"
   ▼
Resolver (ISP / Route 53 Resolver)   ── caches answers (TTL)
   │
   ├─▶ Root servers      → "ask the .com TLD servers"
   ├─▶ TLD (.com) servers→ "ask example.com's name servers"
   └─▶ Authoritative NS  → Route 53 hosted zone returns the record
   │
   ▼
IP address (e.g. 1.2.3.4)  →  browser connects
```

1. The **resolver** checks its **cache**; if expired/missing it walks the hierarchy.
2. **Root** → **TLD (** `.com` **)** → **authoritative name servers** (Route 53 for your zone).
3. Route 53 applies your **routing policy** (+ health checks) and returns the best answer.
4. The resolver **caches** the answer for the record's **TTL** and hands the IP to the browser.

4. 🏷️ Hosted Zones (Public vs Private)

A **hosted zone** holds the records for a domain. When you create one, Route 53 assigns **4 name servers (NS)**.

	Public Hosted Zone	Private Hosted Zone
Visible to	The **public internet**	Only inside **associated VPC(s)**
Use	Public websites, APIs	Internal service names, split-horizon DNS
Resolves from	Anywhere	Resources within the VPC

- For a **registered domain** to use Route 53, set the domain's NS records at the registrar to the **4 NS values** of its public hosted zone (automatic if you register the domain in Route 53).
- **Split-horizon DNS:** same domain name resolves differently for internal (private zone) vs external (public zone) clients.

5. 📇 DNS Record Types

Record	Maps	Example
A	Name → **IPv4** address	`example.com → 1.2.3.4`
AAAA	Name → **IPv6** address	`example.com → 2606:...`
CNAME	Name → **another name** (alias)	`www → example.com`
Alias	Name → an **AWS resource** (Route 53 extension)	`example.com → ALB/CloudFront/S3`
MX	Mail servers (with priority)	`example.com → 10 mail.example.com`
TXT	Arbitrary text (SPF, DKIM, domain verification)	`v=spf1 ...`
NS	Name servers for the zone	delegation
SOA	Start of Authority (zone metadata)	one per zone
SRV	Service location (host + port)	SIP, etc.
PTR	IP → name (reverse DNS)	`4.3.2.1.in-addr.arpa`
CAA	Which CAs may issue certs	`0 issue "amazon.com"`

---

6. ⚖️ Alias vs CNAME

A frequent exam/real-world distinction:

	Alias (Route 53)	CNAME
Points to	**AWS resources** (ALB, CloudFront, S3, API GW, another Route 53 record)	Any **DNS name**
At the **zone apex** (`example.com`)?	✅ Yes	❌ No (DNS forbids CNAME at apex)
Cost	**Free** queries to AWS targets	Charged like a standard query
TTL	Managed by AWS (follows the target)	You set it
Resolves to	The target's current IP(s) automatically	Another name (extra lookup)

> ✅ **Rule of thumb:** pointing at an **AWS resource**? Use an **Alias** — especially at the **root/apex** domain where CNAME isn't allowed. Pointing at an external hostname? Use a **CNAME** on a subdomain.

---

7. 🧭 Routing Policies

Route 53 decides *which* record value to return based on the policy:

Policy	Returns the record based on…	Use case
Simple	One record, no logic	Single resource
Weighted	Split by assigned weights (e.g. 80/20)	A/B tests, gradual rollouts
Latency-based	The region with **lowest latency** to the user	Multi-region apps, fast UX
Failover	Primary if healthy, else secondary	Active-passive DR
Geolocation	The user's **location** (continent/country/state)	Localized content, compliance
Geoproximity	Distance to resources (with bias to expand/shrink)	Traffic-shaping by geography
Multivalue answer	Up to 8 healthy records at random	Simple client-side load spreading
IP-based	The client's IP/CIDR block	Route by known networks/ISPs

> Most policies can be combined with **health checks** so unhealthy endpoints are dropped from answers.

---

8. 🩺 Health Checks & Failover

- **Health checks** probe an endpoint (HTTP/HTTPS/TCP) on an interval; a target is **unhealthy** after N failed checks.
- Types: **endpoint** checks (hit a URL/IP), **calculated** checks (combine other checks), and **CloudWatch alarm** checks (healthy = alarm OK).
- Tie a health check to a record so Route 53 **stops returning** unhealthy endpoints.
- **Failover routing** pairs a **primary** (returned while healthy) with a **secondary** (returned when primary fails) → active-passive disaster recovery.

```
Primary (health check OK)  ─┐
                            ├─▶ Route 53 returns the healthy one
Secondary (standby)        ─┘
```

---

9. 🛒 Domain Registration

- Route 53 can **register and manage domains** (acts as a registrar) for many TLDs — registration auto-creates a matching **public hosted zone**.
- Features: **auto-renew**, **transfer lock**, **privacy protection** (hides WHOIS contact info), and transfers in/out.
- You can also register elsewhere and just **host DNS** in Route 53 by pointing the registrar's NS records to Route 53's 4 name servers.

---

10. 🔗 Route 53 Resolver (Hybrid DNS)

- Inside a VPC, the **Route 53 Resolver** (the `.2` address) answers DNS automatically.
- For **hybrid** setups (cloud ↔ on-prem):
  - **Inbound endpoint** — lets on-prem servers resolve names in your private hosted zones.
  - **Outbound endpoint** + **Resolver rules** — forward queries for specific domains from the VPC to on-prem DNS.
- Enables one consistent DNS namespace across on-prem and AWS.

---

11. ⏱️ TTL & Records Behaviour

- **TTL** controls how long resolvers cache an answer. **High TTL** = fewer queries (cheaper, but slow to change). **Low TTL** = fast propagation (good before a migration), more queries.
- **Lower the TTL ahead of a planned change** (e.g. to 60s) so cutover propagates quickly, then raise it back.
- **Alias** records to AWS targets don't take a manual TTL — they follow the target.
- DNS changes are **eventually consistent** across the internet's resolver caches (bounded by TTL).

---

12. 💰 Pricing

You pay for:

1. **Hosted zones** — per zone per month (first 25 cheaper; more is less each).
2. **Queries** — per million DNS queries (latency/geo/geoproximity cost a bit more than standard).
3. **Health checks** — per check per month (AWS-endpoint vs external, plus optional features like string matching/HTTPS).
4. **Domain registration** — annual fee per domain (varies by TLD).

> 💸 **Cost tips: Alias** queries to AWS targets are **free** (prefer Alias over CNAME for AWS resources) · consolidate records into fewer hosted zones · don't leave health checks running for retired endpoints · use sensible TTLs to limit query volume.

---

13. ⌨️ Common CLI Commands

```
aws route53 list-hosted-zones
aws route53 create-hosted-zone --name example.com --caller-reference $(date +%s)

# Create/update a record via a change batch file
aws route53 change-resource-record-sets --hosted-zone-id Z123 \
  --change-batch file://change.json

aws route53 list-resource-record-sets --hosted-zone-id Z123
aws route53 get-change --id /change/C123            # track propagation status

aws route53 create-health-check --caller-reference hc1 \
  --health-check-config Type=HTTPS,FullyQualifiedDomainName=example.com,Port=443

# Domains (registrar API)
aws route53domains register-domain --domain-name example.com --duration-in-years 1
aws route53domains list-domains
```

```
// change.json — upsert an A-alias to an ALB
{
  "Changes": [{
    "Action": "UPSERT",
    "ResourceRecordSet": {
      "Name": "example.com",
      "Type": "A",
      "AliasTarget": {
        "HostedZoneId": "Z35SXDOTRQ7X7K",
        "DNSName": "my-alb-123.us-east-1.elb.amazonaws.com",
        "EvaluateTargetHealth": true
      }
    }
  }]
}
```

---

14. ✅ Best Practices

- **Use Alias records** for AWS resources (free queries; works at the zone apex).
- **Lower TTLs before migrations**, restore them after, to control propagation speed.
- **Attach health checks** to routing records so traffic avoids unhealthy endpoints.
- **Latency or geolocation routing** for global apps; **failover** for DR.
- **Private hosted zones** for internal names; **split-horizon** when public + private differ.
- Enable **domain auto-renew + transfer lock + privacy** on registered domains.
- Keep DNS as **code** (Terraform/CloudFormation) and review changes — DNS mistakes are high-blast-radius.
- Add **CAA** records to restrict which CAs can issue certificates for your domain.

---

15. 🧠 Quick Mental Model

- **Route 53 = AWS DNS + domain registrar + health checks.** Global, 100% SLA, port 53.
- A **hosted zone** holds your domain's **records**; **public** = internet, **private** = inside a VPC.
- Resolution walks **root** → **TLD** → **your Route 53 name servers**, then caches by **TTL**.
- **Alias** (AWS targets, free, works at apex) vs **CNAME** (any name, not at apex).
- **Routing policies** pick the answer: simple, weighted, latency, failover, geolocation, geoproximity, multivalue, IP-based.
- **Health checks** drop unhealthy endpoints; **failover** gives active-passive DR.
- **Resolver endpoints** bridge DNS between on-prem and VPC (hybrid).
- Lower **TTL** before changes; prefer **Alias** to cut cost; manage DNS **as code**.

---

16. ❓ Common Interview Questions

Rapid-fire questions interviewers ask about Route 53:

- **Q: Alias vs CNAME?** — Alias points to AWS resources, works at the **zone apex**, and queries to AWS targets are free. CNAME points to any name but **can't be used at the apex**.
- **Q: What routing policies exist?** — Simple, Weighted, Latency-based, Failover, Geolocation, Geoproximity, Multivalue answer, IP-based.
- **Q: How does Route 53 do failover?** — Health checks mark endpoints unhealthy; failover routing returns the primary while healthy, else the secondary (active-passive DR).
- **Q: Public vs private hosted zone?** — Public resolves from the internet; private resolves only inside associated VPC(s). Same name in both = split-horizon DNS.
- **Q: What does TTL control?** — How long resolvers cache an answer. Lower it before a migration for fast propagation, then raise it back.
- **Q: How do you point a registered domain at Route 53?** — Set the domain's NS records (at the registrar) to the hosted zone's 4 name servers.
- **Q: Latency vs Geolocation routing?** — Latency routes to the lowest-latency region; Geolocation routes by the user's physical location (compliance/localization).
- **Q: Is Route 53 regional?** — No — it's a **global** service with a 100% availability SLA.

📝 *Notes compiled as a quick reference for Amazon Route 53.*